

5-DAY BUILD PLAN

API Gateway with Go, Redis & Docker

5 Days

Duration

7 Milestones

Checkpoints

4 Services

Containers

1 Goal

Working System

This plan breaks the project into five focused days. Each day has a clear goal, specific tasks with time estimates, and an end-of-day test to confirm you're on track. Follow the order — each day's output is the foundation for the next.

Before You Start — Install These Now

Go 1.22+ · docker + docker-compose · git · VS Code or GoLand

go install github.com/golang-jwt/jwt/v5@latest

go install github.com/redis/go-redis/v9@latest

Verify: go version | docker --version | docker-compose --version

Day 1 — Project Setup & Basic HTTP Server

The only goal today is a working Go HTTP server in a Docker container. No middleware, no proxying — just the skeleton that everything else will attach to. Getting the project structure right now saves pain later.

DAY 1	Project Skeleton & First Server Environment · folder structure · running Go in Docker
0:00–0:30	Create the folder structure Make all directories: api-gateway/, user-service/, product-service/, config/. Init a Go module in each: <code>go mod init github.com/yourname/api-gateway</code>
0:30–1:30	Write the stub microservices user-service/main.go and product-service/main.go — each is a simple net/http server with 2-3 hardcoded JSON endpoints. GET /users returns <code>{{id:1,name:'Alice'}}</code> . These will be your proxy targets.
1:30–2:30	Write the gateway main.go A basic <code>http.ListenAndServe</code> on :8080 with a /health endpoint returning 200 OK and a catch-all handler returning 404. No routing logic yet — just confirm the server starts.
2:30–3:30	Write Dockerfiles for each service Use multi-stage builds: FROM golang:1.22-alpine AS builder → go build → FROM alpine → copy binary. Keeps images small (~15MB). One Dockerfile per service.
3:30–4:30	Write docker-compose.yml Define 4 services: gateway, user-service, product-service, redis. Set <code>depends_on</code> . Expose only port 8080 on gateway. Put all on a shared network called gateway_net.
4:30–5:00	Test end-to-end <code>docker-compose up --build</code> . Hit <code>http://localhost:8080/health</code> → 200. Check logs. Fix any container startup issues.
GOAL	docker-compose up starts all 4 containers. curl localhost:8080/health → 200 OK.

Day 1 Checklist

☐	go mod init done in all 3 service directories
☐	user-service responds on its internal port with JSON
☐	product-service responds on its internal port with JSON
☐	gateway /health returns 200
☐	All 4 containers start with <code>docker-compose up --build</code>
☐	No containers crash on startup (check docker-compose logs)
☐	.gitignore created — includes .env, vendor/, binaries

- ☐ git init + first commit: 'Initial project structure'

Day 1 Tip — Multi-stage Docker builds

Stage 1 (builder): FROM golang:1.22-alpine → COPY go.mod go.sum → go mod download → COPY . . → CGO_ENABLED=0 go build -o /app ./...

Stage 2 (final): FROM alpine:latest → COPY --from=builder /app /app → EXPOSE 8080 → CMD ["/app"]

Result: a ~12MB image instead of a ~800MB one. Always use multi-stage for Go.

Day 1 Key Files to Create

File	Contents
api-gateway/main.go	http.ListenAndServe, /health handler, catch-all 404
user-service/main.go	GET /users and GET /users/{id} with hardcoded JSON
product-service/main.go	GET /products and GET /products/{id} with hardcoded JSON
docker-compose.yml	4 services, shared network, port 8080 exposed, redis image
*/Dockerfile	Multi-stage build for each Go service
.env	GATEWAY_PORT=8080, JWT_SECRET=changeme (never commit this)

Day 2 — Reverse Proxy & Dynamic Routing

Today you connect the gateway to the real microservices using Go's built-in `httputil.ReverseProxy`. By end of day, a request to `/users/*` reaches user-service and `/products/*` reaches product-service — automatically.

DAY 2	Reverse Proxy & Route Table httputil.ReverseProxy · Director function · path routing
0:00–0:45	Read Go's httputil.ReverseProxy docs Understand <code>NewSingleHostReverseProxy</code> , the <code>Director</code> func, and <code>ModifyResponse</code> . You will customise these. Don't skip this — 30 mins of reading saves 2 hours of debugging.
0:45–1:45	Build the route config In <code>proxy/proxy.go</code> , define a route table as a slice of structs: <code>{Prefix string, Target string}</code> . Example: <code>{'/users', 'http://user-service:8001'}</code> , <code>{'/products', 'http://product-service:8002'}</code> . Load targets from environment variables.
1:45–3:00	Implement the proxy handler Write a <code>ProxyHandler</code> function. It iterates the route table, finds the longest matching prefix, creates a <code>ReverseProxy</code> for that target, and calls <code>ServeHTTP</code> . If no match, return 404. Use <code>strings.HasPrefix</code> for matching.
3:00–3:45	Handle path rewriting Decide: does user-service expect <code>/users/42</code> or just <code>/42</code> ? Set the <code>Director</code> func to strip the prefix if needed. Test both options against your stub services before deciding.
3:45–4:30	Wire the proxy into main.go Replace the catch-all 404 handler with the <code>ProxyHandler</code> . The <code>/health</code> route stays separate — do not proxy it.
4:30–5:00	Test all routes <code>curl localhost:8080/users</code> , <code>curl localhost:8080/products/1</code> , <code>curl localhost:8080/nonexistent</code> . Confirm correct responses and correct 404 for unknown paths.
GOAL	Requests to <code>/users/*</code> and <code>/products/*</code> are forwarded to the correct service and return real data.

Day 2 Checklist

☐	<code>curl localhost:8080/users</code> → returns user-service JSON response
☐	<code>curl localhost:8080/products</code> → returns product-service JSON response
☐	<code>curl localhost:8080/nonexistent</code> → returns 404 (not a proxy error)
☐	<code>curl localhost:8080/health</code> → still returns 200 (not proxied)
☐	Target URLs loaded from environment variables, not hardcoded
☐	Route table is a config struct, not an if-else chain

- ☐ Tested path prefix stripping — services receive correct path

Day 2 Tip — Hostname vs localhost inside Docker

WRONG: target = 'http://localhost:8001' → This is the gateway container's localhost, not user-service.

RIGHT: target = 'http://user-service:8001' → Docker Compose resolves service names as hostnames.

If proxy calls are failing with 'connection refused', this is almost certainly why.

Day 2 Tip — Test services directly first

Before connecting via the gateway, curl each service directly from inside Docker:

```
docker-compose exec user-service wget -qO- http://localhost:8001/users
```

If the service doesn't work directly, the proxy can't fix it — solve it at the source.

Day 3 — Logger Middleware & JWT Authentication

Today you implement the two most foundational middleware components. Logger gives you visibility into every request. JWT Auth gates access to your proxied services. By end of day, any request without a valid token is rejected before it reaches a service.

DAY 3	Logger + JWT Auth Middleware Middleware pattern · ResponseWriter wrapping · JWT validation
0:00–1:00	Implement Logger middleware Write middleware/logger.go. Wrap http.ResponseWriter in a custom struct to capture the status code. Record: method, path, client IP, user agent. Start the timer before calling next. Log status + latency after next returns.
1:00–1:30	Wire Logger into the chain Apply it as the outermost middleware so it wraps everything including auth errors. Test: make requests and confirm structured log lines appear for every call, including failed ones.
1:30–2:00	Install JWT library go get github.com/golang-jwt/jwt/v5. Read the library's README for 10 minutes. Understand ParseWithClaims, the MapClaims type, and how to extract custom claims like user_id.
2:00–3:30	Implement Auth middleware Write middleware/auth.go. Extract Bearer token from Authorization header. Parse with your JWT_SECRET. Verify signature and expiry. On success: store user_id in request context via context.WithValue. On failure: write 401 and return immediately.
3:30–4:00	Write a token generator helper Write cmd/gentoken/main.go — a small CLI that takes --user and --secret flags and prints a signed JWT. You'll use this for all your curl tests from now on.
4:00–4:45	Wire Auth into the middleware chain Chain: Logger(Auth(ProxyHandler)). Auth wraps the proxy. Logger wraps everything. The /health endpoint should bypass auth — handle this with a conditional or a separate mux.
4:45–5:00	Test auth flows No token → 401. Wrong token → 401. Expired token → 401. Valid token → 200 with service response. Check that logger logs all outcomes.
GOAL	Only requests with a valid signed JWT reach the microservices. All others get 401.

Day 3 Checklist

☐	Every request produces a log line with method, path, status, and latency (ms)
☐	curl without Authorization header → 401
☐	curl with Authorization: Bearer badtoken → 401

- | | |
|--------------------------|--|
| ☐ | curl with expired token → 401 |
| ☐ | curl with valid token → 200, response from service |
| ☐ | User ID from JWT is stored in request context (verify with a debug log) |
| ☐ | /health endpoint is NOT protected by auth (returns 200 without token) |
| ☐ | JWT_SECRET is read from environment variable, not hardcoded |
| ☐ | gentoken helper works: go run cmd/gentoken/main.go --user=u1 --secret=\$JWT_SECRET |

Day 3 Tip — Never call next() after writing a response

Pattern that causes bugs: if invalid { http.Error(w, '401', 401) } next.ServeHTTP(w, r) ← WRONG

Pattern that works: if invalid { http.Error(w, '401', 401); return } ← CORRECT

After writing any response (including errors), return immediately. Not returning causes 'superfluous response.WriteHeader call' warnings and corrupted responses.

Day 3 Tip — Middleware chaining pattern in Go

```
func Logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // before: start timer, log request
        next.ServeHTTP(w, r)
        // after: log status and latency
    })
}
```

Wrap at startup: handler := Logger(Auth(ProxyHandler))

Day 4 — Redis Rate Limiting

Today you implement the most technically complex feature: distributed rate limiting using a token bucket stored in Redis. By end of day, any user who exceeds their quota gets a 429 response, and the limits are enforced consistently even if you run multiple gateway instances.

DAY 4	Token Bucket Rate Limiting with Redis go-redis · Lua scripts · atomic operations · X-RateLimit headers
0:00–0:45	Install and connect go-redis go get github.com/redis/go-redis/v9. Write a Redis client constructor in config/redis.go that reads REDIS_URL from env and returns a connected *redis.Client. Test the connection on startup with client.Ping(ctx).
0:45–1:30	Design the token bucket data model Each user's bucket is a Redis Hash with fields: tokens (float64), last_refill (Unix timestamp int64). Key format: ratelimit:user:{user_id}. On first request for a new user, initialise with max_tokens tokens and current timestamp.
1:30–3:00	Write the Lua script for atomic refill + consume The script must: read tokens + last_refill, compute elapsed seconds since last_refill, add (elapsed * refill_rate) tokens capped at max_tokens, if tokens >= 1 then deduct 1 and return (1, remaining), else return (0, 0). This all happens atomically — no race conditions.
3:00–4:00	Implement the rate limit middleware Write middleware/ratelimit.go. Get user_id from context (set by auth middleware). Call the Lua script. If allowed: add X-RateLimit-Limit, X-RateLimit-Remaining headers. If blocked: return 429 with Retry-After header showing seconds until next token.
4:00–4:30	Wire RateLimit into the chain Chain becomes: Logger(Auth(RateLimit(ProxyHandler))). Auth must come before RateLimit because RateLimit needs the user_id from context.
4:30–5:00	Test rate limiting behaviour Set limit low (e.g. 5 requests/min) for testing. Send 10 rapid requests with a valid token. First 5 → 200. Next 5 → 429. Wait 1 minute. Try again → 200. Check Redis keys with redis-cli.
GOAL	Per-user rate limits enforced. Requests over the limit get 429 with Retry-After. Limits persist in Redis across gateway restarts.

Day 4 Checklist

☐	Redis connection established on gateway startup (Ping succeeds)
☐	First request from a new user creates a Redis Hash key
☐	Sending N+1 requests where N = limit → N successes + 1 × 429
☐	Response headers include X-RateLimit-Limit and X-RateLimit-Remaining on allowed requests

☐	429 response includes Retry-After header
☐	Bucket refills over time — after waiting, requests succeed again
☐	Verified Redis keys with: <code>docker-compose exec redis redis-cli HGETALL ratelimit:user:u1</code>
☐	Rate limit config (max tokens, refill rate) read from environment variables
☐	Lua script used (not separate GET + SET commands) — verified atomic behaviour

Day 4 Tip — Why a Lua script and not two Redis commands?

Without Lua: GET tokens → compute → SET tokens (3 separate round-trips)

Problem: two concurrent requests both GET '1 token', both compute '0 after deduct', both SET 0 — but two requests went through.

With Lua: Redis runs the entire script in one atomic operation — no concurrent interleaving possible.

Use `client.Eval(ctx, luaScript, []string{key}, args...)` from `go-redis`.

Day 4 Tip — Test Redis manually during development

Open a Redis CLI session while testing: `docker-compose exec redis redis-cli`

Watch keys in real time: `MONITOR` (shows every command hitting Redis)

Inspect a user's bucket: `HGETALL ratelimit:user:u1`

Reset a bucket for retesting: `DEL ratelimit:user:u1`

Day 5 — Polish, Testing & Full Docker Compose

The features are built. Today you make everything production-quality: iron out Docker networking, write tests, add error handling, clean up the code, and verify the complete system works end-to-end. This is also your preparation for the viva/demo.

DAY 5	Integration, Testing & Demo Prep Unit tests · graceful shutdown · error handling · demo script
0:00–1:00	Write unit tests for middleware Create <code>middleware/auth_test.go</code> , <code>middleware/ratelimit_test.go</code> , <code>middleware/logger_test.go</code> . Use <code>httptest.NewRecorder()</code> and <code>httptest.NewRequest()</code> . Test: no token → 401, bad token → 401, valid token → 200, empty bucket → 429. Aim for the happy path + 2 error cases per middleware.
1:00–1:45	Add graceful shutdown to the gateway Replace <code>http.ListenAndServe</code> with a <code>http.Server</code> struct. Use <code>signal.NotifyContext</code> for <code>SIGTERM/SIGINT</code> . Call <code>server.Shutdown(ctx)</code> with a 10-second deadline. This is table stakes for production Go servers.
1:45–2:30	Improve error handling Every proxy error (backend down, timeout) should return a clean 502 Bad Gateway with a JSON body, not an empty response or a Go panic. Set the <code>ReverseProxy</code> 's <code>ErrorHandler</code> field. Add timeouts to the proxy transport (5s dial, 30s response).
2:30–3:00	Add X-Request-ID tracing header In the logger middleware: if the incoming request has an <code>X-Request-ID</code> header, use it. Otherwise, generate a UUID (use <code>github.com/google/uuid</code>). Attach it to the proxied request and the response. Log it with every log line. This enables distributed tracing.
3:00–3:30	Final Docker Compose hardening Add healthcheck directives to each service in <code>docker-compose.yml</code> . Add <code>restart: unless-stopped</code> . Verify <code>depends_on</code> with <code>condition: service_healthy</code> . Test: <code>docker-compose down</code> & <code>docker-compose up</code> — everything should come up cleanly in order.
3:30–4:15	Write the README Cover: project overview, prerequisites, how to run (<code>docker-compose up</code>), how to generate a test token, example curl commands for each feature (auth, rate limit, routing), architecture diagram link, environment variable reference.
4:15–5:00	Full end-to-end demo run Run through the demo script below. Time yourself. Fix anything that breaks. Make sure you can explain every component and every decision.
GOAL	docker-compose up starts cleanly. All 7 features work. You can demo and explain the project confidently.

Day 5 Final Checklist — Ship Criteria

- ☐ `docker-compose up --build` starts all containers without errors

☐	curl localhost:8080/health → 200 (no auth needed)
☐	curl localhost:8080/users (no token) → 401 Unauthorized
☐	curl localhost:8080/users (valid token) → 200 with user data
☐	curl localhost:8080/products (valid token) → 200 with product data
☐	curl localhost:8080/unknown (valid token) → 404 Not Found
☐	10 rapid requests → first N succeed, rest get 429 with Retry-After
☐	Wait for refill → requests succeed again (rate limit resets)
☐	Every request produces a log line with method, path, status, latency, request ID
☐	Stopping a microservice → gateway returns 502, not a crash
☐	docker-compose down && docker-compose up → everything recovers
☐	go test ./... passes with no failures
☐	README explains setup and example curl commands
☐	No hardcoded secrets in any source file

Demo Script (for Viva / Presentation)

Run through this sequence to demonstrate every feature in 5 minutes:

#	Action	What it proves
1	docker-compose up --build	All 4 containers start, Redis connects, services register
2	curl localhost:8080/health	Gateway is alive, /health bypasses auth
3	curl localhost:8080/users (no token)	Auth middleware: rejects unauthenticated requests → 401
4	go run cmd/gentoken/main.go --user=u1	You control token generation — show the JWT structure
5	curl -H 'Authorization: Bearer <token>' localhost:8080/users	Full pipeline works: auth → proxy → user-service response
6	curl with token → localhost:8080/products	Dynamic routing: different path goes to different service
7	Send 15 rapid requests in a loop	Rate limiting: first N succeed, rest get 429 with Retry-After
8	redis-cli HGETALL ratelimit:user:u1	Show Redis state — token count stored, updated in real time
9	docker-compose stop user-service	Error handling: gateway returns 502, doesn't crash

10	docker-compose start user-service	Recovery: service comes back, requests succeed again
----	-----------------------------------	--

Environment Variables Reference

Variable	Example Value	Notes
GATEWAY_PORT	8080	Port the gateway listens on
JWT_SECRET	super-secret-key	NEVER commit this. Use a long random string in prod.
REDIS_URL	redis:6379	Service name, not localhost, inside Docker
USER_SERVICE_URL	http://user-service:8001	Internal Docker hostname
PRODUCT_SERVICE_URL	http://product-service:8002	Internal Docker hostname
RATE_LIMIT_MAX_TOKENS	100	Token bucket capacity per user
RATE_LIMIT_REFILL_RATE	10	Tokens added per second per user
LOG_LEVEL	info	debug / info / warn / error

If You Fall Behind — Triage Guide

If a day runs long, use this priority order. Core features matter more than polish.

Priority	Feature	Can Skip / Simplify?
MUST	HTTP server + Docker Compose	No. Everything runs on this.
MUST	Reverse proxy + dynamic routing	No. Core purpose of the project.
MUST	JWT authentication middleware	No. Required feature.
MUST	Redis rate limiting	No. Required feature.
SHOULD	Logger middleware	Yes — simplify to a one-liner <code>fmt.Printf</code> if short on time.
SHOULD	Unit tests	Yes — skip if needed, mention in viva what you would test.
COULD	Graceful shutdown	Yes — safe to skip for demo.
COULD	X-Request-ID tracing	Yes — bonus feature only.
COULD	502 error handler	Yes — default proxy error is acceptable for demo.

Quick Reference Card

Useful Commands

Command	What it does
<code>docker-compose up --build</code>	Build images and start all containers
<code>docker-compose logs -f gateway</code>	Follow gateway logs in real time
<code>docker-compose exec redis redis-cli</code>	Open Redis CLI inside the container
<code>go test ./...</code>	Run all tests in the project
<code>go build -o gateway ./...</code>	Build the gateway binary locally
<code>curl -i localhost:8080/health</code>	Test health endpoint with response headers
<code>redis-cli HGETALL ratelimit:user:u1</code>	Inspect rate limit state for user u1
<code>redis-cli DEL ratelimit:user:u1</code>	Reset rate limit bucket for testing
<code>redis-cli MONITOR</code>	Watch all Redis commands in real time

5-Day Progress Summary

Day	Focus	End-of-Day Goal	Key Concepts
1	Setup & HTTP Server	docker-compose up starts 4 containers	Docker, multi-stage builds
2	Reverse Proxy & Routing	Requests routed to correct service	httputil.ReverseProxy
3	Logger + JWT Auth	Invalid tokens return 401	Middleware chain, JWTs
4	Redis Rate Limiting	Excess requests return 429	Token bucket, Lua scripts
5	Polish + Tests + Demo	Full demo works, tests pass	httptest, graceful shutdown

Test every milestone before moving to the next. A working foundation beats a broken feature.